一、Base64 编码

Base64 编码将二进制数据编码为可显示的字母和数字，用于传送图形，声音和传真等非文本数据，**常用于 MIME 电子邮件格式**中。其使用含有 65 个字符的 ASCII 字符集，并用 6 个进制位表示一个可显示字符。

表 6.4　Base64 编码表

数　值	编　码	数　值	编　码	数　值	编　码	数　值	编　码
0	A	17	R	34	i	51	z
1	B	18	S	35	j	52	0
2	C	19	T	36	k	53	1
3	D	20	U	37	l	54	2
4	E	21	V	38	m	55	3
5	F	22	W	39	n	56	4
6	G	23	X	40	o	57	5
7	H	24	Y	41	p	58	6
8	I	25	Z	42	q	59	7
9	J	26	a	43	r	60	8
10	K	27	b	44	s	61	9
11	L	28	c	45	t	62	+
12	M	29	d	46	u	63	/
13	N	30	e	47	v		
14	O	31	f	48	w	(pad)	=
15	P	32	g	49	x		
16	Q	33	h	50	y		

image-20241107183531620

- Base64 编码过程：
    - 以 3 个字节为一组(不足 3 位补 0)，将第一个字节放在 24 位缓冲区的高 8 位，第二个放在中 8 位，第三个放在低 8 位。
    - 对 24 位缓冲区以 6 位为索引，高位优先，从字符串“ABCDEFGHIJKLMN OPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/”中取出相应元素作为输出
    - “ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/”即为字符集

二、动态调试

IDA 的静态分析功能已经足够强大，很多时候只需要 IDA 的静态就足够了，那为什么还需要动态调试呢。

2.1 与静态分析相互验证

之前讲过，在指令集正确的情况下，IDA 中 100%可信的只有汇编代码，而汇编命令到 C 代码是不唯一的，IDA 中的伪 C 代码只能够参考，不能完全相信，这时候我们可以通过动态调试进行验证。

2.2 绕过复杂解密逻辑

这里引入一个 Windows API，VirtualProtect，具体怎么使用这里不多讲，只需要知道这个是用来修改内存页的权限的，经常用于做自修改代码，如果在程序中看到 VirtualProtect 的字样，大概率是在做自修改。这里简单讲一下自修改代码。简单来说，就是把一个函数的机器码加密存储到原来函数所在的内存中，等到程序运行的时候将内存解密为原函数。如果这个时候你纯静态分析，会耗费很长时间，并且解出来的机器码还需要翻译为 C 语言，但是只需要下断点动态调试，就可以直接恢复函数。

2.3 IDA 动态调试

开始调试

今天只讲 Windows 的调试方式，选择 Local Windows debugger，点击左侧绿色三角

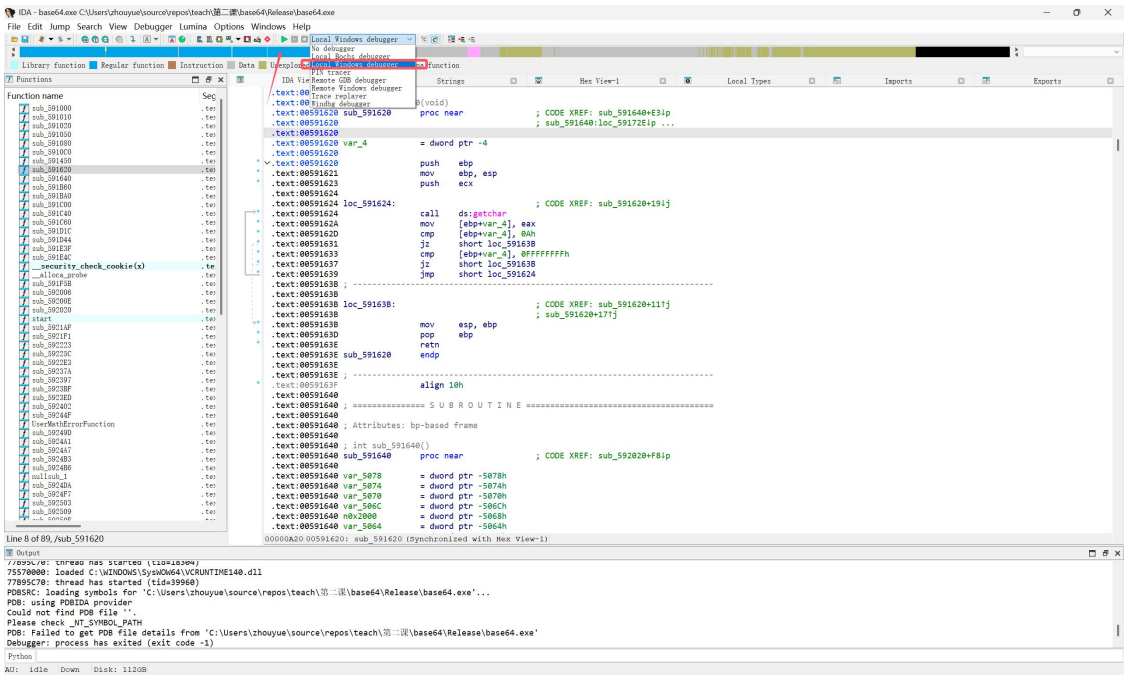
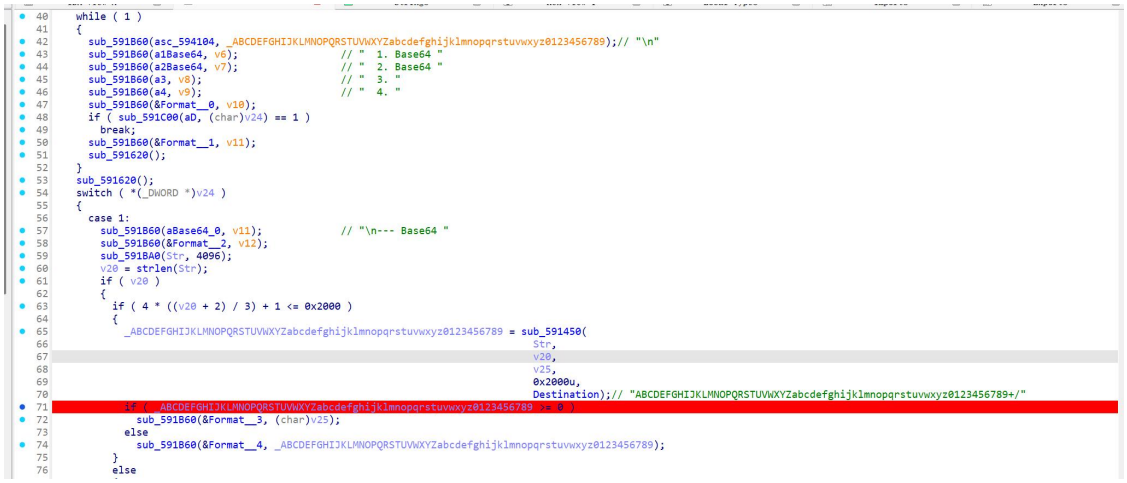


image-20260706140922762

这时会跳出警告，点击 yes 即可。

下断点



```
40  while ( 1 )
41  {
42    sub_591B60(asc_594104, _ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789);// "\n"
43    sub_591B60(a1Base64, v6);                 // "  1. Base64 "
44    sub_591B60(a2Base64, v7);                 // "  2. Base64 "
45    sub_591B60(a3, v8);                       // "  3. "
46    sub_591B60(a4, v9);                       // "  4. "
47    sub_591B60(&Format__0, v10);
48    if ( sub_591C00(aD, (char)v24) == 1 )
49      break;
50    sub_591B60(&Format__1, v11);
51    sub_591620();
52  }
53  sub_591620();
54  switch ( *(_DWORD *)v24 )
55  {
56    case 1:
57      sub_591B60(aBase64_0, v11);             // "\n--- Base64 "
58      sub_591B60(&Format__2, v12);
59      sub_591BA0(Str, 4096);
60      v20 = strlen(Str);
61      if ( v20 )
62      {
63        if ( 4 * ((v20 + 2) / 3) + 1 <= 0x2000 )
64        {
65          _ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789 = sub_591450(
66                                                    Str,
67                                                    v20,
68                                                    v25,
69                                                    0x2000u,
70                                                    Destination);// "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
71          if ( _ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789 >= 0 )
72            sub_591B60(&Format__3, (char)v25);
73          else
74            sub_591B60(&Format__4, _ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789);
75        }
76        else
```

image-20260706141202684

比如这里我想下断点在 base64 编码完成后的第一行代码处，我就将光标移到对应代码行，按 F2 下断点。

在入门阶段的绝大多数逆向分析题中，掌握普通断点已经足够。

常用快捷键

继续运行：F9

单步步入：F7

单步步过：F8

执行到返回：Ctrl +F7

执行到光标处：F4

添加、删除断点：F2